Guía de Respuestas Correctas
Taller 1: Invariante de Bucle
ECIN-608 – Diseño y Análisis de Algoritmos

Guía de Estudio Académica

2do Semestre, 2026

Índice

1. Instrucciones Generales para Obtener los 100 Puntos **5**

2. Algoritmo 1: Búsqueda Lineal **6**
- 2.1. Pseudocódigo . . . 6
- 2.2. Invariante de Bucle . . . 6
- 2.3. Demostración . . . 6
  - 2.3.1. Inicialización . . . 6
  - 2.3.2. Mantenimiento . . . 6
  - 2.3.3. Terminación . . . 6
- 2.4. Complejidad . . . 7
- 2.5. Casos Límite . . . 7

3. Algoritmo 2: Contar Ocurrencias de x **8**
- 3.1. Pseudocódigo . . . 8
- 3.2. Invariante de Bucle . . . 8
- 3.3. Demostración . . . 8
  - 3.3.1. Inicialización . . . 8
  - 3.3.2. Mantenimiento . . . 8
  - 3.3.3. Terminación . . . 8
- 3.4. Complejidad . . . 8
- 3.5. Casos Límite . . . 9

4. Algoritmo 3: Encontrar el Máximo **10**
- 4.1. Pseudocódigo . . . 10
- 4.2. Invariante de Bucle . . . 10
- 4.3. Demostración . . . 10
  - 4.3.1. Inicialización . . . 10
  - 4.3.2. Mantenimiento . . . 10
  - 4.3.3. Terminación . . . 10
- 4.4. Complejidad . . . 10
- 4.5. Casos Límite . . . 11

5. Algoritmo 4: Encontrar el Mínimo **12**
- 5.1. Pseudocódigo . . . 12
- 5.2. Invariante de Bucle . . . 12
- 5.3. Demostración . . . 12
  - 5.3.1. Inicialización . . . 12

5.3.2. Mantenimiento 12
5.3.3. Terminación 12
5.4. Complejidad 12
5.5. Casos Límite 12

6. Algoritmo 5: Verificar Elementos Positivos **13**
6.1. Pseudocódigo 13
6.2. Invariante de Bucle 13
6.3. Demostración 13
6.3.1. Inicialización 13
6.3.2. Mantenimiento 13
6.3.3. Terminación 13
6.4. Complejidad 13
6.5. Casos Límite 13

7. Algoritmo 6: Verificar Arreglo Ordenado (No Decreciente) **14**
7.1. Pseudocódigo 14
7.2. Invariante de Bucle 14
7.3. Demostración 14
7.3.1. Inicialización 14
7.3.2. Mantenimiento 14
7.3.3. Terminación 14
7.4. Complejidad 14
7.5. Casos Límite 14

8. Algoritmo 7: Suma de un Arreglo **15**
8.1. Pseudocódigo 15
8.2. Invariante de Bucle 15
8.3. Demostración 15
8.3.1. Inicialización 15
8.3.2. Mantenimiento 15
8.3.3. Terminación 15
8.4. Complejidad 15
8.5. Casos Límite 15

9. Algoritmo 8: Invertir un Arreglo **16**
9.1. Pseudocódigo 16
9.2. Invariante de Bucle 16
9.3. Demostración 16
9.3.1. Inicialización 16
9.3.2. Mantenimiento 16
9.3.3. Terminación 16
9.4. Complejidad 16
9.5. Casos Límite 16

10. Algoritmo 9: Máximo Subarreglo (Kadane) **17**
10.1. Pseudocódigo 17
10.2. Invariante de Bucle 17
10.3. Demostración 17
10.3.1. Inicialización 17
10.3.2. Mantenimiento 17
10.3.3. Terminación 17

10.4. Complejidad 17
10.5. Casos Límite 18

11. Algoritmo 10: Suma Máxima de Ventana Fija **19**
11.1. Pseudocódigo 19
11.2. Invariante de Bucle 19
11.3. Demostración 19
11.3.1. Inicialización 19
11.3.2. Mantenimiento 19
11.3.3. Terminación 19
11.4. Complejidad 19
11.5. Casos Límite 20

12. Algoritmo 11: Bubble Sort **21**
12.1. Pseudocódigo 21
12.2. Invariante de Bucle 21
12.3. Demostración 21
12.3.1. Inicialización 21
12.3.2. Mantenimiento (Bucle Interno) 21
12.3.3. Mantenimiento (Bucle Externo) 21
12.3.4. Terminación 21
12.4. Complejidad 21
12.5. Casos Límite 22

13. Algoritmo 12: Insertion Sort **23**
13.1. Pseudocódigo 23
13.2. Invariante de Bucle 23
13.3. Demostración 23
13.3.1. Inicialización 23
13.3.2. Mantenimiento 23
13.3.3. Terminación 23
13.4. Complejidad 23
13.5. Casos Límite 24

14. Algoritmo 13: Selection Sort **25**
14.1. Pseudocódigo 25
14.2. Invariante de Bucle 25
14.3. Demostración 25
14.3.1. Inicialización 25
14.3.2. Mantenimiento 25
14.3.3. Terminación 25
14.4. Complejidad 26
14.5. Casos Límite 26

15. Algoritmo 14: Merge Sort **27**
15.1. Pseudocódigo 27
15.2. Invariante de Bucle 27
15.3. Demostración 27
15.3.1. Inicialización (MergeSort — Caso base) 27
15.3.2. Inicialización (Merge) 27
15.3.3. Mantenimiento (Merge) 28
15.3.4. Terminación (Merge) 28

15.3.5. Terminación (MergeSort) 28
15.4. Complejidad 28
15.5. Casos Límite 28

16. Algoritmo 15: Quick Sort **29**
16.1. Pseudocódigo 29
16.2. Invariante de Bucle 29
16.3. Demostración 29
16.3.1. Inicialización 29
16.3.2. Mantenimiento 29
16.3.3. Terminación 30
16.4. Complejidad 30
16.5. Casos Límite 30

17. Algoritmo 16: Two-Sum (Par con Suma Objetivo) **31**
17.1. Pseudocódigo 31
17.2. Invariante de Bucle 31
17.3. Demostración 31
17.3.1. Inicialización 31
17.3.2. Mantenimiento 31
17.3.3. Terminación 31
17.4. Complejidad 32
17.5. Casos Límite 32

18. Resumen de Complejidades **33**

1. Instrucciones Generales para Obtener los 100 Puntos

Para cada algoritmo, la presentación debe demostrar **competencia** en las siguientes áreas según la rúbrica:

- **Contenido técnico (30 pts):** Algoritmo correcto, invariante formal, inicialización, mantenimiento, terminación, complejidad y casos límite.
- **Estructura y apoyo visual (20 pts):** 8–10 diapositivas, legibles, con diagramas/animaciones.
- **Comunicación oral (20 pts):** Explicación clara, vocabulario preciso, no leer diapositivas.
- **Preguntas y dominio (20 pts):** Respuestas fundamentadas, reconocer limitaciones.
- **Tiempo y participación (10 pts):** 5–7 minutos, ambos participantes activos.

Estructura Obligatoria para Cada Algoritmo

1. **Pseudocódigo** del algoritmo completo.
2. **Definición formal del invariante** $I(i)$ con notación matemática.
3. **Demostración de inicialización**: $I(i_0)$ verdadero.
4. **Demostración de mantenimiento**: $I(i) \Rightarrow I(i+1)$.
5. **Demostración de terminación**: $I(i_{\text{final}}) \wedge \neg\text{cond_bucle} \Rightarrow$ postcondición.
6. **Análisis de complejidad**: temporal (mejor, peor, promedio) y espacial.
7. **Casos límite**: discusión de al menos 4 casos especiales.

2. Algoritmo 1: Búsqueda Lineal

2.1. Pseudocódigo

Algorithm 1 Búsqueda Lineal

Require: Arreglo $A[1..n]$, elemento objetivo x
Ensure: Índice j tal que $A[j] = x$, o -1 si no existe

```
1: j ← −1
2: for i ← 1 to n do
3:     if A[i] = x then
4:         j ← i
5:         return j
6:     end if
7: end for
8: return j
```

2.2. Invariante de Bucle

Sea el bucle `for` que recorre i desde 1 hasta n. El invariante es:

$$\boxed{I(i) \equiv \text{“No existe } k \in \{1, \ldots, i-1\} \text{ tal que } A[k] = x\text{”}}$$

Es decir, antes de examinar $A[i]$, se ha verificado que x no aparece en las posiciones 1 a $i - 1$.

2.3. Demostración

2.3.1. Inicialización

Antes de la primera iteración, $i = 1$. El conjunto $\{1, \ldots, 0\}$ es vacío, por lo que la afirmación “no existe $k \in \emptyset$ tal que $A[k] = x$” es **vacuamente verdadera**. □

2.3.2. Mantenimiento

Supongamos $I(i)$ verdadero al inicio de la iteración i:

- Si $A[i] = x$: el algoritmo retorna i y el bucle termina. No hay iteración siguiente, por lo que la propiedad se mantiene trivialmente.
- Si $A[i] \neq x$: al inicio de la iteración $i + 1$, el conjunto $\{1, \ldots, i\}$ no contiene x (pues $I(i)$ afirma que no está en $\{1, \ldots, i-1\}$ y acabamos de verificar $A[i] \neq x$). Luego $I(i+1)$ es verdadero. □

2.3.3. Terminación

El bucle termina en dos casos:

1. **Retorno anticipado** ($A[i] = x$): Se retorna i con $A[i] = x$, lo cual es correcto.
2. **Bucle completa todas las iteraciones**: Al finalizar, $i = n + 1$. El invariante $I(n+1)$ indica que $x \notin A[1..n]$. Se retorna -1, lo cual es correcto. □

2.4. Complejidad

- **Mejor caso**: $O(1)$ — el objetivo está en la primera posición.
- **Peor caso**: $O(n)$ — el objetivo está en la última posición o no existe.
- **Promedio**: $O(n)$ — en promedio se recorre $n/2$ elementos.
- **Espacial**: $O(1)$ — solo se usan variables auxiliares constantes.

2.5. Casos Límite

1. **Arreglo vacío** ($n = 0$): El bucle no se ejecuta, se retorna -1. Correcto.
2. **Un solo elemento**: Se compara $A[1]$ con x. Si coincide retorna 1, si no retorna -1.
3. **Elemento repetido**: Retorna la *primera* aparición (índice menor).
4. **Objetivo fuera de rango**: Se recorre todo el arreglo y se retorna -1.
5. **Todos los elementos iguales al objetivo**: Retorna 1 (primera posición).

3. Algoritmo 2: Contar Ocurrencias de x

3.1. Pseudocódigo

Algorithm 2 Contar Ocurrencias

Require: Arreglo $A[1..n]$, valor x
Ensure: Cantidad de veces que x aparece en A

```
1: c ← 0
2: for i ← 1 to n do
3:     if A[i] = x then
4:         c ← c + 1
5:     end if
6: end for
7: return c
```

3.2. Invariante de Bucle

$$\boxed{I(i) \equiv c = |\{k \in \{1, \ldots, i-1\} : A[k] = x\}|}$$

Es decir, c contiene el número de apariciones de x en las posiciones ya recorridas (1 a $i-1$).

3.3. Demostración

3.3.1. Inicialización

Cuando $i = 1$, el conjunto $\{1, \ldots, 0\} = \emptyset$. Su cardinalidad es 0 y $c = 0$. Luego $I(1)$ es verdadero. □

3.3.2. Mantenimiento

Supongamos $I(i)$ verdadero. En la iteración i:

- Si $A[i] = x$: c se incrementa en 1. Al inicio de la iteración $i+1$, $c = |\{k \leq i : A[k] = x\}|$, que es exactamente $I(i+1)$.
- Si $A[i] \neq x$: c no cambia. Como $I(i)$ contaba correctamente $\{k \leq i-1 : A[k] = x\}$ y $A[i] \neq x$, el conteo para $\{k \leq i : A[k] = x\}$ es el mismo. □

3.3.3. Terminación

Al terminar el bucle, $i = n + 1$. Por el invariante:

$$c = |\{k \in \{1, \ldots, n\} : A[k] = x\}|$$

Que es exactamente la cantidad de apariciones de x en $A[1..n]$. □

3.4. Complejidad

- **Todos los casos**: $O(n)$ — siempre se recorre todo el arreglo.
- **Espacial**: $O(1)$.

3.5. Casos Límite

1. **Arreglo vacío**: $c = 0$. Correcto.
2. x **no aparece**: $c = 0$. Correcto.
3. **Todos los elementos son** x: $c = n$.
4. **Un solo elemento**: $c \in \{0, 1\}$ según coincida o no.

4. Algoritmo 3: Encontrar el Máximo

4.1. Pseudocódigo

Algorithm 3 Encontrar el Máximo

Require: Arreglo $A[1..n]$, $n \geq 1$
Ensure: Elemento máximo de A

```
1: max ← A[1]
2: for i ← 2 to n do
3:     if A[i] > max then
4:         max ← A[i]
5:     end if
6: end for
7: return max
```

4.2. Invariante de Bucle

$$\boxed{I(i) \equiv \text{max} = \text{máx}\{A[1], A[2], \ldots, A[i-1]\}}$$

El valor de `max` es el máximo entre los elementos de las posiciones 1 a $i-1$.

4.3. Demostración

4.3.1. Inicialización

Cuando $i = 2$: $\text{max} = A[1] = \text{máx}\{A[1]\}$. El conjunto $\{A[1]\}$ tiene máximo $A[1]$. □

4.3.2. Mantenimiento

Supongamos $I(i)$ verdadero: $\text{max} = \text{máx}\{A[1], \ldots, A[i-1]\}$.

- Si $A[i] > \text{max}$: se actualiza $\text{max} \leftarrow A[i]$. Entonces $\text{max} = \text{máx}\{A[1], \ldots, A[i]\}$, que es $I(i+1)$.
- Si $A[i] \leq \text{max}$: no hay cambio. Como $\text{max} \geq A[i]$ y $\text{max} = \text{máx}\{A[1], \ldots, A[i-1]\}$, entonces $\text{max} = \text{máx}\{A[1], \ldots, A[i]\}$. □

4.3.3. Terminación

Cuando el bucle termina, $i = n + 1$. Por el invariante:

$$\text{max} = \text{máx}\{A[1], \ldots, A[n]\}$$

Que es el máximo del arreglo completo. □

4.4. Complejidad

- **Todos los casos**: $O(n)$ — se recorre el arreglo completo siempre.
- **Espacial**: $O(1)$.

4.5. Casos Límite

1. **Un solo elemento**: El bucle no se ejecuta, retorna $A[1]$.
2. **Todos iguales**: max nunca se actualiza, retorna $A[1]$.
3. **Máximo al final**: Se actualiza en la última iteración.
4. **Máximo al inicio**: max nunca se actualiza.
5. **Valores negativos**: Funciona correctamente.

5. Algoritmo 4: Encontrar el Mínimo

5.1. Pseudocódigo

Algorithm 4 Encontrar el Mínimo

Require: Arreglo $A[1..n]$, $n \geq 1$
Ensure: Elemento mínimo de A

```
1: min ← A[1]
2: for i ← 2 to n do
3:     if A[i] < min then
4:         min ← A[i]
5:     end if
6: end for
7: return min
```

5.2. Invariante de Bucle

$$\boxed{I(i) \equiv \text{min} = \text{mín}\{A[1], A[2], \ldots, A[i-1]\}}$$

5.3. Demostración

La demostración es análoga a la del Algoritmo 3, reemplazando máx por mín y la condición $>$ por $<$.

5.3.1. Inicialización

$i = 2$: $\text{min} = A[1] = \text{mín}\{A[1]\}$. $\square$

5.3.2. Mantenimiento

Si $A[i] < \text{min}$, se actualiza y $\text{min} = \text{mín}\{A[1], \ldots, A[i]\}$. Si $A[i] \geq \text{min}$, el mínimo no cambia y la propiedad se preserva. $\square$

5.3.3. Terminación

$i = n + 1$: $\text{min} = \text{mín}\{A[1], \ldots, A[n]\}$. $\square$

5.4. Complejidad

- **Todos los casos**: $O(n)$.
- **Espacial**: $O(1)$.

5.5. Casos Límite

1. **Un solo elemento**: Retorna $A[1]$.
2. **Todos iguales**: Retorna $A[1]$.
3. **Mínimo al inicio**: min nunca se actualiza.
4. **Valores negativos**: Funciona correctamente.

6. Algoritmo 5: Verificar Elementos Positivos

6.1. Pseudocódigo

Algorithm 5 Verificar Todos Positivos

Require: Arreglo $A[1..n]$
Ensure: **true** si todos los elementos son estrictamente positivos, **false** en caso contrario

```
1: todos_pos ← true
2: for i ← 1 to n do
3:     if A[i] ≤ 0 then
4:         todos_pos ← false
5:         return false
6:     end if
7: end for
8: return true
```

6.2. Invariante de Bucle

$$\boxed{I(i) \equiv \forall k \in \{1, \ldots, i-1\} : A[k] > 0}$$

Todos los elementos visitados hasta la posición $i-1$ son estrictamente positivos.

6.3. Demostración

6.3.1. Inicialización

$i = 1$: El conjunto $\{1, \ldots, 0\} = \emptyset$. La afirmación "$\forall k \in \emptyset : A[k] > 0$" es **vacuamente verdadera**. □

6.3.2. Mantenimiento

Supongamos $I(i)$ verdadero. Si $A[i] \leq 0$, el algoritmo retorna **false** y no hay siguiente iteración. Si $A[i] > 0$, entonces $\forall k \in \{1, \ldots, i\} : A[k] > 0$, lo que es $I(i+1)$. □

6.3.3. Terminación

Si el bucle completa todas las iteraciones, $i = n+1$ y $\forall k \in \{1, \ldots, n\} : A[k] > 0$. Se retorna **true**. Si se retorna **false** antes, es porque se encontró $A[i] \leq 0$. □

6.4. Complejidad

- **Mejor caso**: $O(1)$ — el primer elemento no es positivo.
- **Peor caso**: $O(n)$ — todos positivos o el último no lo es.
- **Espacial**: $O(1)$.

6.5. Casos Límite

1. **Arreglo vacío**: El bucle no se ejecuta, retorna **true** (vacuamente verdadero).
2. **Contiene cero**: Retorna **false** (se requiere *estrictamente* positivo).
3. **Todos negativos**: Retorna **false** en la primera iteración.
4. **Un solo elemento positivo**: Retorna **true**.

7. Algoritmo 6: Verificar Arreglo Ordenado (No Decreciente)

7.1. Pseudocódigo

Algorithm 6 Verificar Orden No Decreciente

Require: Arreglo $A[1..n]$
Ensure: **true** si $A[1] \leq A[2] \leq \cdots \leq A[n]$

```
1: ordenado ← true
2: for i ← 1 to n − 1 do
3:     if A[i] > A[i + 1] then
4:         ordenado ← false
5:         return false
6:     end if
7: end for
8: return true
```

7.2. Invariante de Bucle

$$\boxed{I(i) \equiv \forall k \in \{1, \ldots, i-1\} : A[k] \leq A[k+1]}$$

Todas las parejas consecutivas visitadas hasta la posición i están en orden no decreciente.

7.3. Demostración

7.3.1. Inicialización

$i = 1$: No se ha verificado ninguna pareja. La afirmación es vacuamente verdadera. □

7.3.2. Mantenimiento

Si $A[i] \leq A[i+1]$, la propiedad se extiende a la posición i. Si $A[i] > A[i+1]$, se retorna **false**. En el caso de continuación, $I(i+1)$ se mantiene. □

7.3.3. Terminación

Si el bucle completa todas las iteraciones ($i = n$), entonces $\forall k \in \{1, \ldots, n-1\} : A[k] \leq A[k+1]$, lo que define un arreglo no decreciente. □

7.4. Complejidad

- **Mejor caso**: $O(1)$ — la primera pareja viola el orden.
- **Peor caso**: $O(n)$ — el arreglo está ordenado.
- **Espacial**: $O(1)$.

7.5. Casos Límite

1. **Arreglo vacío o un elemento**: Retorna **true** (trivialmente ordenado).
2. **Todos iguales**: Retorna **true** (no decreciente incluye empates).
3. **Estrictamente decreciente**: Retorna **false** en la primera iteración.
4. **Elementos duplicados**: Son permitidos en orden no decreciente.

8. Algoritmo 7: Suma de un Arreglo

8.1. Pseudocódigo

Algorithm 7 Suma de Arreglo

Require: Arreglo $A[1..n]$
Ensure: $\sum_{k=1}^{n} A[k]$

```
1: s ← 0
2: for i ← 1 to n do
3:     s ← s + A[i]
4: end for
5: return s
```

8.2. Invariante de Bucle

$$\boxed{I(i) \equiv s = \sum_{k=1}^{i-1} A[k]}$$

8.3. Demostración

8.3.1. Inicialización

$i = 1$: $s = 0 = \sum_{k=1}^{0} A[k] = 0$ (suma vacía). □

8.3.2. Mantenimiento

En la iteración i: $s \leftarrow s + A[i]$. Si antes $s = \sum_{k=1}^{i-1} A[k]$, después $s = \sum_{k=1}^{i} A[k]$, que es $I(i+1)$. □

8.3.3. Terminación

$i = n + 1$: $s = \sum_{k=1}^{n} A[k]$, que es la suma total. □

8.4. Complejidad

- **Todos los casos**: $O(n)$.
- **Espacial**: $O(1)$.

8.5. Casos Límite

1. **Arreglo vacío**: Retorna 0.
2. **Todos negativos**: La suma es negativa, correcto.
3. **Un solo elemento**: Retorna $A[1]$.
4. **Overflow**: Discutir límites del tipo de dato.

9. Algoritmo 8: Invertir un Arreglo

9.1. Pseudocódigo

Algorithm 8 Invertir Arreglo in-place

Require: Arreglo $A[1..n]$
Ensure: A con sus elementos en orden inverso

```
1: for i ← 1 to ⌊n/2⌋ do
2:     temp ← A[i]
3:     A[i] ← A[n − i + 1]
4:     A[n − i + 1] ← temp
5: end for
```

9.2. Invariante de Bucle

Definimos las posiciones simétricas: para cada j, la posición espejo es $n - j + 1$.

$$I(i) \equiv \begin{cases} \forall k \in \{1, \ldots, i-1\} : A[k] = B[n-k+1] & \text{(izquierda: ya trae el valor original de la derecha)} \\ \forall k \in \{n-i+2, \ldots, n\} : A[k] = B[n-k+1] & \text{(derecha: ya trae el valor original de la izquierda)} \end{cases}$$

donde B es el arreglo original. Alternativamente:

$I(i) \equiv$ "Los primeros $i-1$ y los últimos $i-1$ elementos están intercambiados correctamente."

9.3. Demostración

9.3.1. Inicialización

$i = 1$: No se ha intercambiado ningún elemento. El arreglo es igual al original. Verdadero. □

9.3.2. Mantenimiento

En la iteración i, se intercambian $A[i]$ y $A[n-i+1]$. Si antes los primeros $i-1$ y últimos $i-1$ estaban invertidos, ahora los primeros i y últimos i también lo están. □

9.3.3. Terminación

Cuando $i = \lfloor n/2 \rfloor + 1$, todos los pares simétricos $(A[k], A[n-k+1])$ han sido intercambiados para $k = 1, \ldots, \lfloor n/2 \rfloor$. El arreglo está completamente invertido. □

9.4. Complejidad

- **Todos los casos**: $O(n)$ — se realizan $\lfloor n/2 \rfloor$ intercambios.
- **Espacial**: $O(1)$ — in-place con una variable temporal.

9.5. Casos Límite

1. **Arreglo vacío**: No se ejecuta ningún intercambio.
2. **Un solo elemento**: No se ejecuta ningún intercambio.
3. **Par de elementos**: Se intercambian una vez.
4. **Número impar de elementos**: El elemento central no se mueve.
5. **Palíndromo**: El resultado es igual al original.

10. Algoritmo 9: Máximo Subarreglo (Kadane)

10.1. Pseudocódigo

Algorithm 9 Máximo Subarreglo (Kadane)

Require: Arreglo $A[1..n]$
Ensure: Mayor suma de un subarreglo contiguo

```
1: max_actual ← A[1]
2: max_global ← A[1]
3: for i ← 2 to n do
4:     max_actual ← máx(A[i], max_actual + A[i])
5:     if max_actual > max_global then
6:         max_global ← max_actual
7:     end if
8: end for
9: return max_global
```

10.2. Invariante de Bucle

$$I(i) \equiv \begin{cases} \text{max_global} = \text{máx}_{1 \le j \le k \le i} \sum_{t=j}^{k} A[t] \\ \text{max_actual} = \text{máx}_{1 \le j \le i} \sum_{t=j}^{i} A[t] \text{ (máximo subarreglo terminando en } i) \end{cases}$$

10.3. Demostración

10.3.1. Inicialización

$i = 1$: max_actual $= A[1] =$ máx subarreglo terminando en 1. max_global $= A[1] =$ máx subarreglo en $A[1.,1]$. □

10.3.2. Mantenimiento

En la iteración i, el subarreglo máximo terminando en i es:

- $A[i]$ solo (reiniciar subarreglo), o
- $A[i]$ concatenado al mejor subarreglo terminando en $i-1$.

La operación max_actual $\leftarrow$ máx$(A[i], \text{max_actual} + A[i])$ captura exactamente esta decisión. Luego `max_global` se actualiza si `max_actual` es mayor. □

10.3.3. Terminación

Al finalizar, `max_global` contiene la máxima suma de cualquier subarreglo contiguo en $A[1..n]$. □

10.4. Complejidad

- **Todos los casos**: $O(n)$ — una pasada por el arreglo.
- **Espacial**: $O(1)$.

10.5. Casos Límite

1. **Un solo elemento**: Retorna $A[1]$.
2. **Todos negativos**: Retorna el menos negativo (el máximo).
3. **Todos positivos**: Retorna la suma de todo el arreglo.
4. **Mezcla**: El algoritmo maneja correctamente cambios de signo.

11. Algoritmo 10: Suma Máxima de Ventana Fija

11.1. Pseudocódigo

Algorithm 10 Suma Máxima de Ventana Fija (tamaño k)

Require: Arreglo $A[1..n]$, tamaño de ventana $k \leq n$
Ensure: Mayor suma entre ventanas contiguas de tamaño k

```
1: s ← ∑_{j=1}^{k} A[j]                          ▷ Suma de la primera ventana
2: max_suma ← s
3: for i ← k + 1 to n do
4:     s ← s + A[i] − A[i − k]                    ▷ Deslizar ventana
5:     if s > max_suma then
6:         max_suma ← s
7:     end if
8: end for
9: return max_suma
```

11.2. Invariante de Bucle

$$I(i) \equiv s = \sum_{j=i-k}^{i-1} A[j] \quad \wedge \quad \text{max_suma} = \max_{t=k}^{i-1} \sum_{j=t-k+1}^{t} A[j]$$

s es la suma de la ventana que termina en la posición $i-1$, y `max_suma` es la mayor suma de todas las ventanas ya procesadas.

11.3. Demostración

11.3.1. Inicialización

Antes del bucle: $s = \sum_{j=1}^{k} A[j]$ (primera ventana). max_suma $= s$. Correcto para la primera ventana. □

11.3.2. Mantenimiento

En la iteración i: la ventana se desliza una posición. Se suma $A[i]$ (nuevo elemento) y se resta $A[i-k]$ (elemento que sale). La nueva suma s corresponde a la ventana $[i-k+1, i]$. Si $s >$ max_suma, se actualiza. □

11.3.3. Terminación

Al finalizar, todas las ventanas de tamaño k fueron evaluadas y `max_suma` contiene la mayor suma. □

11.4. Complejidad

- **Todos los casos**: $O(n)$ — cada elemento entra y sale de la ventana una vez.
- **Espacial**: $O(1)$.

11.5. Casos Límite

1. $k = n$: Una sola ventana, retorna la suma de todo el arreglo.
2. $k = 1$: Retorna el máximo elemento.
3. **Todos negativos**: Retorna la suma menos negativa.
4. $n = k$: Solo hay una ventana.

12. Algoritmo 11: Bubble Sort

12.1. Pseudocódigo

Algorithm 11 Bubble Sort

Require: Arreglo $A[1..n]$
Ensure: A ordenado en orden no decreciente

```
1: for i ← 1 to n − 1 do
2:     for j ← 1 to n − i do
3:         if A[j] > A[j + 1] then
4:             intercambiar(A[j], A[j + 1])
5:         end if
6:     end for
7: end for
```

12.2. Invariante de Bucle

Para el bucle externo (índice i):

$$I(i) \equiv \begin{cases} \text{Los últimos } i-1 \text{ elementos de } A \text{ están en su posición final y son los } i-1 \text{ mayores.} \\ A[j] \leq A[k] \text{ para todo } j > n-i \text{ y } k > j. \end{cases}$$

Para el bucle interno (índice j):

$I_{\text{int}}(j) \equiv A[j] = \text{máx}\{A[1], \ldots, A[j]\}$ o el mayor ha sido "burbujeado" hacia la derecha.

12.3. Demostración

12.3.1. Inicialización

$i = 1$: No se han colocado elementos finales. El invariante es trivialmente verdadero. □

12.3.2. Mantenimiento (Bucle Interno)

Durante la iteración j del bucle interno, el mayor elemento de $A[1..n-i+1]$ "burbujea" hacia la posición $n-i+1$. Después del bucle interno, $A[n-i+1]$ es el elemento en su posición correcta. □

12.3.3. Mantenimiento (Bucle Externo)

Después de la iteración i, los últimos i elementos están ordenados y en sus posiciones finales. □

12.3.4. Terminación

Cuando $i = n$, los últimos $n-1$ elementos están en posición final, lo que implica que todo el arreglo está ordenado. □

12.4. Complejidad

- **Mejor caso**: $O(n)$ — con optimización (bandera `swapped`), arreglo ya ordenado.
- **Peor caso**: $O(n^2)$ — arreglo en orden inverso.
- **Promedio**: $O(n^2)$.
- **Espacial**: $O(1)$ — in-place.

12.5. Casos Límite

1. **Arreglo vacío o un elemento**: No se ejecuta ningún intercambio.
2. **Ya ordenado**: Con optimización, $O(n)$; sin ella, $O(n^2)$.
3. **Orden inverso**: Peor caso, cada par se intercambia.
4. **Elementos duplicados**: El operador $>$ garantiza estabilidad (no intercambia iguales).

13. Algoritmo 12: Insertion Sort

13.1. Pseudocódigo

Algorithm 12 Insertion Sort

Require: Arreglo $A[1..n]$
Ensure: A ordenado en orden no decreciente

```
1: for i ← 2 to n do
2:     clave ← A[i]
3:     j ← i − 1
4:     while j ≥ 1 ∧ A[j] > clave do
5:         A[j + 1] ← A[j]
6:         j ← j − 1
7:     end while
8:     A[j + 1] ← clave
9: end for
```

13.2. Invariante de Bucle

Para el bucle externo (i):

$$\boxed{I(i) \equiv \text{El subarreglo } A[1..i-1] \text{ está ordenado en orden no decreciente.}}$$

Para el bucle interno (j):

$I_{\text{int}}(j) \equiv$ Todos los elementos de $A[j{+}2..i]$ son mayores que $A[j{+}1]$ y el subarreglo se mantiene ordenado.

13.3. Demostración

13.3.1. Inicialización

$i = 2$: El subarreglo $A[1.,1]$ contiene un solo elemento, que trivialmente está ordenado. □

13.3.2. Mantenimiento

Supongamos $A[1..i-1]$ está ordenado. Se inserta $A[i] =$ clave en su posición correcta:

- Los elementos mayores que `clave` se desplazan a la derecha.
- `clave` se inserta en la posición $j+1$ donde $A[j] \leq$ `clave` $\leq A[j+1]$.
- El subarreglo $A[1..i]$ queda ordenado. □

13.3.3. Terminación

Cuando el bucle termina, $i = n + 1$. El subarreglo $A[1..n]$ está ordenado. □

13.4. Complejidad

- **Mejor caso**: $O(n)$ — arreglo ya ordenado, cada inserción es $O(1)$.
- **Peor caso**: $O(n^2)$ — arreglo en orden inverso, cada inserción mueve todos los elementos.
- **Promedio**: $O(n^2)$.
- **Espacial**: $O(1)$ — in-place.

13.5. Casos Límite

1. **Arreglo vacío o un elemento**: No se ejecuta el bucle.
2. **Ya ordenado**: $O(n)$ — las inserciones son triviales.
3. **Orden inverso**: Peor caso, cada elemento se mueve hasta el inicio.
4. **Estabilidad**: Insertion Sort es estable (no cambia el orden de elementos iguales).

14. Algoritmo 13: Selection Sort

14.1. Pseudocódigo

Algorithm 13 Selection Sort

Require: Arreglo $A[1..n]$
Ensure: A ordenado en orden no decreciente

```
1: for i ← 1 to n − 1 do
2:     min_idx ← i
3:     for j ← i + 1 to n do
4:         if A[j] < A[min_idx] then
5:             min_idx ← j
6:         end if
7:     end for
8:     intercambiar(A[i], A[min_idx])
9: end for
```

14.2. Invariante de Bucle

$$I(i) \equiv \begin{cases} A[1..i-1] \text{ está ordenado.} \\ \forall k \in \{1,\ldots,i-1\}, \forall m \in \{i,\ldots,n\} : A[k] \leq A[m] \end{cases}$$

Los primeros $i-1$ elementos están ordenados y son todos menores o iguales a los elementos restantes.

14.3. Demostración

14.3.1. Inicialización

$i = 1$: El subarreglo $A[1.,0]$ es vacío, trivialmente ordenado. La condición de dominancia es vacuamente verdadera. □

14.3.2. Mantenimiento

En la iteración i:

1. Se busca el mínimo en $A[i..n]$ y se obtiene su índice `min_idx`.
2. Se intercambia $A[i]$ con $A[\text{min_idx}]$.
3. Ahora $A[i]$ es el menor de $A[i..n]$.
4. Dado que $A[1..i-1]$ estaba ordenado y todos sus elementos son $\leq A[i]$ (por la búsqueda del mínimo), $A[1..i]$ está ordenado. □

14.3.3. Terminación

Cuando $i = n$, $A[1..n-1]$ está ordenado y todos sus elementos son $\leq A[n]$. Por transitividad, A está completamente ordenado. □

14.4. Complejidad

- **Todos los casos**: $O(n^2)$ — siempre se busca el mínimo en el subarreglo restante.
- **Espacial**: $O(1)$ — in-place.
- **Intercambios**: Como máximo $n - 1$ intercambios (ventaja sobre Bubble Sort).

14.5. Casos Límite

1. **Arreglo vacío o un elemento**: No se ejecuta.
2. **Ya ordenado**: Sigue siendo $O(n^2)$, pero solo $n - 1$ intercambios.
3. **Orden inverso**: $O(n^2)$ y todos los intercambios son necesarios.
4. **No estable**: Selection Sort no es estable en general (el intercambio puede cambiar el orden de elementos iguales).

15. Algoritmo 14: Merge Sort

15.1. Pseudocódigo

Algorithm 14 Merge Sort

Require: Arreglo $A[1..n]$
Ensure: A ordenado en orden no decreciente

```
1: procedure MergeSort(A, l, r)
2:     if l < r then
3:         m ← ⌊(l + r)/2⌋
4:         MergeSort(A, l, m)
5:         MergeSort(A, m + 1, r)
6:         Merge(A, l, m, r)
7:     end if
8: end procedure

9: procedure Merge(A, l, m, r)
10:    Crear L ← A[l..m], R ← A[m + 1..r]
11:    i ← 1, j ← 1, k ← l
12:    while i ≤ |L| ∧ j ≤ |R| do
13:        if L[i] ≤ R[j] then
14:            A[k] ← L[i], i ← i + 1
15:        else
16:            A[k] ← R[j], j ← j + 1
17:        end if
18:        k ← k + 1
19:    end while
20:    Copiar elementos restantes de L o R en A
21: end procedure
```

15.2. Invariante de Bucle

Para el procedimiento `MergeSort` (recursión):

$I_{\text{merge}} \equiv$ "Después de `MergeSort`(A, l, r), el subarreglo $A[l..r]$ está ordenado."

Para el bucle del procedimiento `Merge`:

$$I(i,j) \equiv \begin{cases} A[l..k-1] \text{ contiene los } k-l \text{ menores elementos de } L[1..i-1] \cup R[1..j-1] \text{ en orden.} \\ \text{L[i..—L—]}y\text{R[j..—R—]}\textit{contienenloselementosnoprocesados.} \end{cases}$$

15.3. Demostración

15.3.1. Inicialización (MergeSort — Caso base)

Si $l \geq r$, el subarreglo tiene 0 o 1 elemento, que está trivialmente ordenado. □

15.3.2. Inicialización (Merge)

Al inicio del bucle, $k = l$, $i = 1$, $j = 1$. $A[l..k-1] = A[l..l-1] = \emptyset$. Verdadero vacuamente. □

15.3.3. Mantenimiento (Merge)

En cada iteración, se selecciona el menor entre $L[i]$ y $R[j]$. Este se coloca en $A[k]$ en su posición correcta relativa. Los elementos restantes conservan su orden. □

15.3.4. Terminación (Merge)

Cuando el bucle termina, uno de los arreglos se agotó. Se copian los elementos restantes del otro (que ya están ordenados). $A[l..r]$ contiene todos los elementos en orden. □

15.3.5. Terminación (MergeSort)

Por inducción: si `MergeSort` funciona para arreglos de tamaño $< n$, entonces después de ordenar $A[l..m]$ y $A[m+1..r]$, `Merge` combina dos arreglos ordenados en uno ordenado. □

15.4. Complejidad

- **Mejor caso**: $O(n \log n)$.
- **Peor caso**: $O(n \log n)$ — siempre divide en dos mitades.
- **Promedio**: $O(n \log n)$.
- **Espacial**: $O(n)$ — se requieren arreglos auxiliares para `Merge`.

15.5. Casos Límite

1. **Arreglo vacío**: La recursión no se ejecuta.
2. **Un solo elemento**: Caso base, no se hace nada.
3. **Ya ordenado**: Sigue siendo $O(n \log n)$.
4. **Todos iguales**: Sigue siendo $O(n \log n)$.
5. **Estabilidad**: Merge Sort es estable si la fusión preserva el orden de elementos iguales ($L[i] \leq R[j]$ en lugar de $<$).

16. Algoritmo 15: Quick Sort

16.1. Pseudocódigo

Algorithm 15 Quick Sort

Require: Arreglo $A[1..n]$
Ensure: A ordenado en orden no decreciente

```
1: procedure QuickSort(A, l, r)
2:     if l < r then
3:         p ← Partition(A, l, r)
4:         QuickSort(A, l, p − 1)
5:         QuickSort(A, p + 1, r)
6:     end if
7: end procedure

8: procedure Partition(A, l, r)
9:     pivote ← A[r]
10:    i ← l − 1
11:    for j ← l to r − 1 do
12:        if A[j] ≤ pivote then
13:            i ← i + 1
14:            intercambiar(A[i], A[j])
15:        end if
16:    end for
17:    intercambiar(A[i + 1], A[r])
18:    return i + 1
19: end procedure
```

16.2. Invariante de Bucle

Para el procedimiento `Partition` (bucle con índice j):

$$I(j) \equiv \begin{cases} \forall k \in \{l, \ldots, i\} : A[k] \leq \text{pivote} \\ \forall k \in \{i+1, \ldots, j-1\} : A[k] > \text{pivote} \end{cases}$$

Todos los elementos a la izquierda de i son $\leq$ pivote, y todos entre $i+1$ y $j-1$ son $>$ pivote.

16.3. Demostración

16.3.1. Inicialización

$j = l$: $i = l - 1$. Los rangos $\{l, \ldots, l-1\}$ y $\{l, \ldots, l-1\}$ son vacíos. Verdadero vacuamente. □

16.3.2. Mantenimiento

En la iteración j:

- Si $A[j] \leq$ pivote: Se incrementa i e intercambia $A[i]$ con $A[j]$. Ahora $A[i] \leq$ pivote y la invariant se mantiene.
- Si $A[j] >$ pivote: No se hace nada. $A[j]$ queda en la región de elementos $>$ pivote. □

16.3.3. Terminación

Al finalizar el bucle, se intercambia $A[i+1]$ con $A[r]$ (el pivote). Ahora:

- $A[l..i] \leq$ pivote
- $A[i+1] =$ pivote
- $A[i+2..r] >$ pivote

El pivote está en su posición final $p = i + 1$. Las llamadas recursivas ordenan los subarreglos izquierdo y derecho. □

16.4. Complejidad

- **Mejor caso**: $O(n \log n)$ — partición equilibrada.
- **Peor caso**: $O(n^2)$ — arreglo ya ordenado o inverso ordenado con pivote en extremo.
- **Promedio**: $O(n \log n)$.
- **Espacial**: $O(\log n)$ — profundidad promedio de la pila de recursión.

16.5. Casos Límite

1. **Arreglo vacío o un elemento**: No se ejecuta partición.
2. **Todos iguales**: Sin optimización, peor caso $O(n^2)$. Con `Dutch National Flag` o pivote aleatorio, $O(n)$.
3. **Ya ordenado**: Peor caso con pivote en el extremo. Mejorar con pivote aleatorio.
4. **No estable**: Quick Sort no es estable en su implementación in-place.
5. **Pivote aleatorio**: Seleccionar un pivote aleatorio evita el peor caso determinístico.

17. Algoritmo 16: Two-Sum (Par con Suma Objetivo)

17.1. Pseudocódigo

Algorithm 16 Two-Sum (con tabla hash)

Require: Arreglo $A[1..n]$, objetivo x
Ensure: Índices (p, q) con $p < q$ tales que $A[p] + A[q] = x$, o -1 si no existe

```
1: H ← ∅                                          ▷ Tabla hash: valor → índice
2: for i ← 1 to n do
3:     if x − A[i] ∈ H then
4:         return (H[x − A[i]], i)
5:     end if
6:     H[A[i]] ← i
7: end for
8: return −1
```

17.2. Invariante de Bucle

$$\boxed{I(i) \equiv H = \{(A[k], k) : 1 \leq k \leq i-1\} \ \wedge \ \forall j, k \in \{1, \ldots, i-1\}, j < k : A[j] + A[k] \neq x}$$

La tabla H contiene exactamente los elementos ya procesados (con su índice), y ningún par de entre esos elementos suma el objetivo x.

17.3. Demostración

17.3.1. Inicialización

Cuando $i = 1$, $H = \emptyset$ y el conjunto de pares procesados es vacío. Ambas condiciones son **vacuamente verdaderas**. □

17.3.2. Mantenimiento

Supongamos $I(i)$ verdadero al inicio de la iteración i. Se inspecciona $A[i]$:

- Si $x - A[i] \in H$: existe $k = H[x - A[i]] < i$ con $A[k] = x - A[i]$, luego $A[k] + A[i] = x$. El algoritmo retorna (k, i), que es correcto. No hay iteración siguiente, por lo que el invariante se mantiene trivialmente.
- Si $x - A[i] \notin H$: para todo $k < i$ se tiene $A[k] + A[i] \neq x$. Como $I(i)$ afirma que ningún par de $\{1, \ldots, i-1\}$ suma x, ningún par de $\{1, \ldots, i\}$ suma x. Tras insertar $H[A[i]] \leftarrow i$, la tabla contiene los elementos de $\{1, \ldots, i\}$, que es $I(i+1)$. □

17.3.3. Terminación

El bucle termina en dos casos:

1. **Retorno anticipado**: se encontró un par (k, i) con $A[k] + A[i] = x$, lo cual es correcto.
2. **Bucle completa todas las iteraciones**: $I(n+1)$ afirma que ningún par de $A[1..n]$ suma x. Se retorna -1, lo cual es correcto. □

Nótese que si existe un par solución (p, q) con $p < q$, en la iteración q la tabla ya contiene $A[p]$ (pues $p < q$), por lo que el algoritmo necesariamente lo encuentra.

17.4. Complejidad

- **Todos los casos**: $O(n)$ esperado — una pasada por el arreglo con operaciones de tabla hash $O(1)$ esperado.
- **Espacial**: $O(n)$ — la tabla hash almacena hasta n entradas.

17.5. Casos Límite

1. **Arreglo vacío o de un elemento**: El bucle no encuentra pares y retorna -1.
2. **Duplicados**: Si $A = [3, 3]$ con $x = 6$, en $i = 2$ se encuentra $3 \in H$ y se retorna $(1, 2)$. El algoritmo nunca usa el mismo elemento dos veces, pues la verificación precede a la inserción.
3. **Objetivo** $x = 0$: Un par de opuestos $A = [-3, 2, 3]$ con $x = 0$ retorna $(1, 3)$.
4. **Valores negativos**: El complemento $x - A[i]$ puede ser negativo y se maneja correctamente con la tabla hash.
5. **Múltiples pares solución**: Retorna el primero que se encuentra al recorrer el arreglo.

18. Resumen de Complejidades

Algoritmo	**Mejor**	**Peor**	**Promedio**
1. Búsqueda Lineal	$O(1)$	$O(n)$	$O(n)$
2. Contar Ocurrencias	$O(n)$	$O(n)$	$O(n)$
3. Encontrar el Máximo	$O(n)$	$O(n)$	$O(n)$
4. Encontrar el Mínimo	$O(n)$	$O(n)$	$O(n)$
5. Verificar Positivos	$O(1)$	$O(n)$	$O(n)$
6. Verificar Ordenado	$O(1)$	$O(n)$	$O(n)$
7. Suma de Arreglo	$O(n)$	$O(n)$	$O(n)$
8. Invertir Arreglo	$O(n)$	$O(n)$	$O(n)$
9. Máx. Subarreglo (Kadane)	$O(n)$	$O(n)$	$O(n)$
10. Suma Máx. Ventana Fija	$O(n)$	$O(n)$	$O(n)$
11. Bubble Sort	$O(n)^*$	$O(n^2)$	$O(n^2)$
12. Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
13. Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
14. Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
15. Quick Sort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$
16. Two-Sum	$O(n)$	$O(n)$	$O(n)$

Cuadro 1: Complejidades temporales. (*) Con optimización de bandera.

Algoritmo	**Espacial**	**Estable**
1–10	$O(1)$	N/A
11. Bubble Sort	$O(1)$	Sí
12. Insertion Sort	$O(1)$	Sí
13. Selection Sort	$O(1)$	No
14. Merge Sort	$O(n)$	Sí
15. Quick Sort	$O(\log n)$	No
16. Two-Sum	$O(n)$	N/A

Cuadro 2: Complejidad espacial y estabilidad.